



Parking Lot Management System

Using Core Java & Object-Oriented Programming Concepts

The picture can't be displayed.

Project Details

Subject: Object Oriented Programming with Java

Project ID: RU-100-01-00012

The picture can't be displayed.

Student Information

Name: Harsh Vardhan

ERP ID: RU-25-10556

College: Rungta International Skills University

Department: Computer Science & Engineering

Academics Year: 2025-29 (First Year)

Introduction

Addressing the growing demand for smart and efficient urban mobility solutions.

Problems in Traditional Systems

Manual parking slot management

Time-consuming vehicle tracking

Difficulty in calculating parking fees

Lack of real-time slot availability

Human errors and inefficiency

Poor customer experience

Why Automation Matters



Efficient slot allocation



Automated fee calculation



Real-time availability tracking



Reduced manual workload



Improved accuracy



Better customer satisfaction

Objectives & Scope

Key goals and application areas for the smart parking system.

Objectives

Manage parking slots efficiently

Assign parking spaces automatically

Track vehicle entry and exit

Calculate parking fees accurately

Reduce manual work and human errors

Apply OOP concepts in Java implementation

Scope



Suitable for malls and shopping centers



Applicable to office buildings and corporate parks



Ideal for colleges and educational institutions



Effective for small to medium parking areas



Can be expanded for larger applications



Scalable architecture for future enhancements

System Requirements & Technologies

Essential hardware, software, and core technologies.

Hardware Requirements



4GB RAM minimum



Intel i5 or equivalent



500MB free storage



1024×768 display or higher

Software Requirements

1

Java JDK 8 or higher

Required for compiling and running the application.

2

Code Editor / IDE

VS Code, Eclipse, or IntelliJ IDEA for development.

3

Operating System

Compatible with Windows, Linux, or macOS environments.

4

Git Version Control

Used for source control and project collaboration.

Technologies Used

Core Java

Object-oriented foundation for the full system.

OOP Concepts

Encapsulation, inheritance, and polymorphism.

Collections

ArrayList and the Collections Framework for data handling.

File I/O

Persistent storage and retrieval of parking data.

Exception Handling

Improves stability and error management.

System Workflow & Process Flow

Simple, sequential steps from start to finish.



Start

Display Menu

User Chooses

Process
Tasks

Store Info

The workflow moves step by step through a continuous cycle, returning users to the main menu whenever they choose to continue.

Class Diagram & OOP Concepts

Minimalist class structure and core OOP principles.

Class Structure (UML-style)

Vehicle Class:

```
- vehicleId: String
- vehicleName: String
- entryTime: LocalDateTime
- exitTime: LocalDateTime
+ getVehicleDetails()
+ calculateDuration()
```

ParkingLot Class:

```
- totalSlots: int
- availableSlots: int
- vehicles: ArrayList<Vehicle>
+ parkVehicle()
+ removeVehicle()
+ searchVehicle()
+ getAvailableSlots()
```

Main Class:

```
- displayMenu()
- handleUserInput()
- main()
```

OOP Concepts Explained

The picture can't be displayed.

Encapsulation: Data hiding with private variables and public methods

The picture can't be displayed.

Abstraction: Hiding complex implementation details

The picture can't be displayed.

Inheritance: Extending classes for code reuse

The picture can't be displayed.

Polymorphism: Method overriding and overloading

PROJECT SCREENSHOT

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  class Vehicle {
5      String vehicleNumber;
6      String vehicleType;
7
8      Vehicle(String vehicleNumber, String vehicleType) {
9          this.vehicleNumber = vehicleNumber;
10         this.vehicleType = vehicleType;
11     }
12 }
13
14 public class ParkingLotManagement {
15
16     static ArrayList<Vehicle> parkingLot = new ArrayList<>();
17
18     // Check if vehicle type is valid
19     public static boolean isValidVehicleType(String type) {
20         return type.equalsIgnoreCase(anotherString: "Bike")
21             || type.equalsIgnoreCase(anotherString: "Car")
22             || type.equalsIgnoreCase(anotherString: "Bus")
23             || type.equalsIgnoreCase(anotherString: "Truck");
24     }
25
26     // Check duplicate vehicle number
27     public static boolean vehicleExists(String number) {
28         for (Vehicle v : parkingLot) {
29             if (v.vehicleNumber.equalsIgnoreCase(number)) {
30                 return true;
31             }
32         }
33         return false;
34     }
35
36     public static void parkVehicle(String number, String type) {
37
```

```
164
165     } else {
166         searchVehicle(number);
167     }
168
169     break;
170
171 case 3:
172     System.out.print($: "Enter Vehicle Number: ");
173     number = sc.nextLine().trim();
174
175     if (number.isEmpty()) {
176         System.out.println(X: "\n!! Enter a Valid Vehicle Number.\n");
177     } else {
178         removeVehicle(number);
179     }
180
181     break;
182
183 case 4:
184     displayStatus();
185     break;
186
187 case 5:
188     System.out.println(X: "\nThank You For Using Parking lot Management System!");
189     sc.close();
190     System.exit(status: 0);
191     break;
192
193 default:
194     System.out.println(X: "\n!! Invalid Choice. Please Try Again.\n");
195
196 }
197
198 }
```

Conclusion & Future Scope

A polished summary of today's results and the next wave of improvements.

This final section highlights the parking system's current strengths, then previews the features that can make it even smarter, faster, and more connected in the future.

Key Achievements

Streamlined parking operations

Organized the core workflow for parking, searching, status checks, and vehicle removal.

Reduced manual effort

Automated repetitive steps to save time and minimize human error.

Improved slot tracking

Made space availability easier to monitor and manage in real time.

Applied Java OOP concepts

Used an object-oriented structure that supports clean code and maintainability.

Created a user-friendly interface

Designed simple screens that make common operations quick and intuitive.

Built a scalable foundation

Prepared the project for future features, integrations, and growth.

Future Enhancements



Online booking system

Enable advance reservations for better convenience and planning.



QR code entry

Speed up access with contactless verification at entry and exit.



Payment gateway integration

Support secure digital payments and faster checkout.



Mobile application

Extend system access to phones for anytime, anywhere use.



Real-time monitoring

Provide live visibility into parking availability and status.



Analytics and notifications

Add reports, insights, and alerts to improve decision-making.

Thank You